

Programação Orientada a Objetos

Interfaces de Orientação a objeto e Java Collections

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



- 1 Interfaces na Orientação a Objetos
- 2 Introdução ao Java Collections Framework
- 3 Hierarquia das Collections

- 4 A Interface List
- 5 LinkedList

Interfaces na Orientação a Objetos

O que é uma Interface?

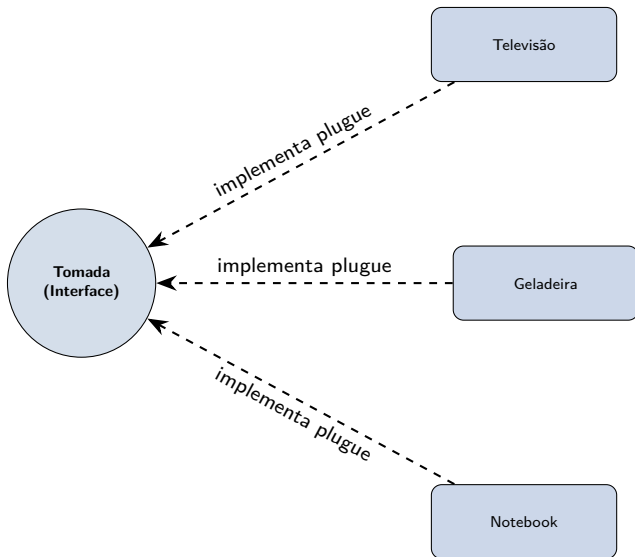
- Em Programação Orientada a Objetos, uma **Interface** é um **contrato** que define um conjunto de comportamentos (métodos) que uma classe deve implementar.
- Uma interface dita **o que** um objeto deve fazer, mas não **como** ele faz.
- Em Java, até a versão 7, interfaces continham apenas assinaturas de métodos (sem corpo) e constantes.
- Serve como uma fronteira de comunicação entre diferentes componentes de um sistema.



- Pense em uma **tomada elétrica** na parede.
- Ela define um "padrão"(Interface): possui furos em posições específicas e fornece energia de uma certa voltagem.
- Qualquer aparelho (TV, Geladeira, Carregador) que tenha um **plugue** compatível com esse padrão pode se conectar e funcionar.
- A tomada não precisa saber **qual** aparelho está conectado, apenas que ele segue o contrato (encaixa no plugue).



Diagrama Conceitual: Analogia



Por que usar Interfaces?

- **Desacoplamento:** O código que usa a interface não fica dependente da implementação específica de uma classe.
- **Polimorfismo:** Permite tratar objetos de classes diferentes de maneira uniforme, desde que implementem a mesma interface.
- **Múltipla Herança de Tipos:** No Java, uma classe só pode herdar (`extends`) de uma única superclasse, mas pode implementar (`implements`) múltiplas interfaces simultaneamente.
- **Padronização:** Facilita o trabalho em equipe, pois o arquiteto pode definir as interfaces (contratos) e os programadores criam as implementações de forma independente.



- Utilizamos a palavra-chave `interface` em vez de `class`.
- Os métodos declarados são implicitamente `public` e `abstract` (não precisam da palavra-chave).

```
1 // Definindo uma interface
2 public interface Autenticavel {
3
4     // Metodo sem corpo (apenas a assinatura)
5     boolean autenticar(String senha);
6
7     // Outro metodo sem corpo
8     void fazerLogout();
9 }
10
```

Implementando uma Interface

- Uma classe assina o contrato usando a palavra-chave `implements`.
- **Regra de Ouro:** A classe concreta **deve** implementar (fornecer o corpo) todos os métodos definidos na interface.

```
1 // A classe assina o contrato "Autenticavel"
2 public class Gerente implements Autenticavel {
3     private String senhaDoGerente = "1234";
4
5     @Override
6     public boolean autenticar(String senha) {
7         return this.senhaDoGerente.equals(senha);
8     }
9
10    @Override
11    public void fazerLogout() {
12        System.out.println("Gerente saiu do sistema.");
13    }
14 }
15
```

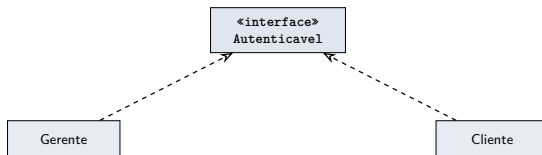
Múltiplas Implementações

Diferentes classes podem implementar a mesma interface de maneiras totalmente diferentes:

```
1 public class Cliente implements Autenticavel {
2     private String tokenApp;
3
4     @Override
5     public boolean autenticar(String senha) {
6         // Logica diferente: usa token de aplicativo
7         return validarTokenNoServidor(this.tokenApp, senha);
8     }
9
10    @Override
11    public void fazerLogout() {
12        System.out.println("Sessao do cliente encerrada.");
13    }
14
15    private boolean validarTokenNoServidor(String t, String s) {
16        return true; // logica simulada
17    }
18 }
19
```

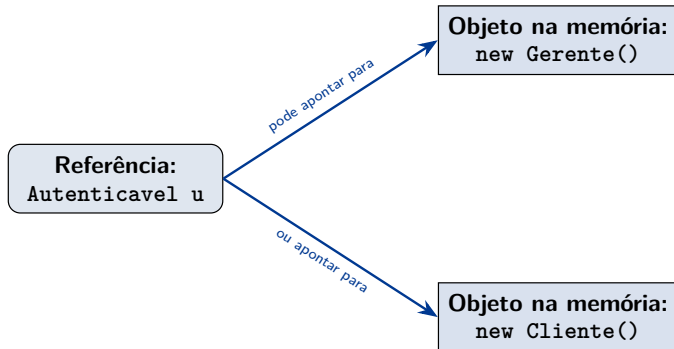


Diagrama UML: Interface vs Implementação



- No diagrama UML, a relação de implementação é representada por uma seta tracejada com a ponta vazada apontando para a interface.

- Programar para a interface, não para a implementação.
- Podemos usar o tipo da Interface como o tipo de uma variável de referência.



- Isso nos permite criar métodos genéricos que aceitam **qualquer** objeto que implemente a interface, sem precisar saber a sua classe exata em tempo de compilação.

Característica	Classe Abstrata	Interface
Herança	Uma classe só pode herdar (extends) de uma classe.	Uma classe pode implementar (implements) múltiplas interfaces.
Métodos	Pode conter métodos abstratos e métodos normais (com corpo).	Métodos sem corpo (Java 8+ permite métodos default e static).
Atributos	Pode ter variáveis de instância normais (com estado).	Apenas constantes (public static final implícitos).
Foco	Relação de identidade (" É um ").	Relação de comportamento (" Faz um ").

Exemplo Prático: Polimorfismo

```
1 public class SistemaLogin {
2     // Aceita QUALQUER objeto que seja Autenticavel
3     public void fazerLogin(Autenticavel usuario, String senha) {
4         if (usuario.autenticar(senha)) {
5             System.out.println("Acesso Liberado!");
6         } else {
7             System.out.println("Acesso Negado!");
8         }
9     }
10
11     public static void main(String[] args) {
12         SistemaLogin sistema = new SistemaLogin();
13         Gerente g1 = new Gerente();
14         Cliente c1 = new Cliente();
15
16         // O mesmo metodo lida com objetos diferentes!
17         sistema.fazerLogin(g1, "1234");
18         sistema.fazerLogin(c1, "senhaErrada");
19     }
20 }
21
```



Introdução ao Java Collections Framework

Motivação: O Problema dos Arrays

- Arrays são a estrutura de dados mais básica e fundamental no Java.
- No entanto, eles apresentam desafios práticos no dia a dia.
- Exemplo de declaração e uso:

```
1 // Criando um array para armazenar 100 nomes
2 String[] nomes = new String[100];
3 nomes[0] = "Alice";
4 nomes[1] = "Bob";
5
6 // E se precisarmos adicionar 101 elementos?
7 // E se precisarmos remover "Alice" e reorganizar?
8
```



- **Tamanho fixo:** O tamanho do array é definido no momento da sua criação. Ele não pode crescer ou encolher dinamicamente conforme a necessidade.
- **Inserções e remoções difíceis:** Remover um elemento do meio do array ou inserir em uma posição específica exige o deslocamento manual dos demais elementos.
- **Pouca flexibilidade:** Não há métodos prontos nativos nos arrays para busca avançada, ordenação fácil ou manipulações complexas (exige implementação própria ou uso da classe utilitária Arrays).



- Para resolver as limitações dos arrays, a linguagem oferece o **Java Collections Framework**.
- **O que é?** É uma arquitetura unificada para representar e manipular coleções de objetos de forma padronizada.
- Ele fornece **interfaces** (que definem o comportamento esperado) e **classes concretas** (que implementam as estruturas de dados).



- **Estruturas dinâmicas:** As coleções crescem e diminuem automaticamente conforme elementos são adicionados ou removidos.
- **Métodos prontos:** Fornece dezenas de métodos úteis e otimizados para inserção, remoção, busca, ordenação e iteração.
- **Melhor reutilização de código:** Padrões de projeto bem definidos garantem que os desenvolvedores usem uma interface comum e intercambiável.
- **Maior produtividade:** Reduz o tempo de desenvolvimento, pois elimina a necessidade de criar estruturas de dados do zero.

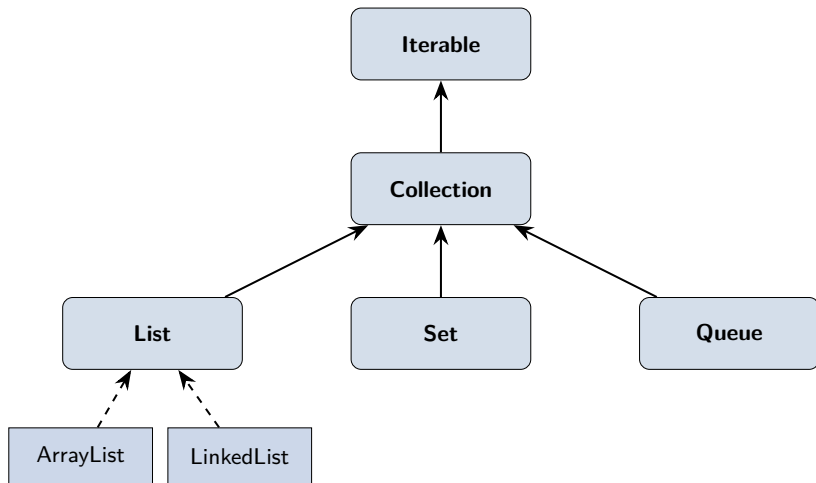


Hierarquia das Collections

- O JCF é composto por um conjunto de **Interfaces** que organizam o comportamento das coleções.
- As coleções em Java são divididas em duas ramificações principais:
 - As coleções baseadas na interface `Collection` (elementos individuais).
 - As coleções baseadas na interface `Map` (pares chave-valor).
- As classes de implementação (como `ArrayList`, `HashSet`, `HashMap`) fornecem as estruturas reais usadas no código.



Diagrama da Hierarquia: Collection



A Interface Collection

- Collection é a interface **principal** da qual a maioria das coleções herda.
- Ela garante que qualquer coleção no Java suporte métodos essenciais:

```
1 public interface Collection<E> extends Iterable<E> {  
2     boolean add(E e);           // Adiciona um elemento  
3     boolean remove(Object o);   // Remove um elemento  
4     boolean contains(Object o); // Verifica existencia  
5     int size();                 // Retorna o tamanho  
6     boolean isEmpty();          // Verifica se esta vazia  
7     void clear();               // Limpa a colecao  
8     Iterator<E> iterator();     // Para percorrer  
9 }  
10
```

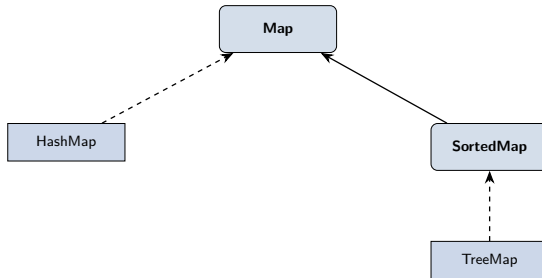


Onde entra o Map?

- **Importante:** A interface Map **não herda** de Collection.
- **Por que?** Diferente das outras coleções que armazenam elementos simples, um Map armazena elementos no formato de **chave** → **valor**.
- Apesar disso, o Map é considerado parte integral do Java Collections Framework.
- Mapas fornecem visualizações (views) baseadas em Collection de seus dados, como `keySet()` (chaves) e `values()` (valores).



Diagrama da Hierarquia: Map



Collection (Exemplo com List):

```
1 List<String> cidades = new ArrayList<>();  
2 cidades.add("Rio Verde");  
3 cidades.add("Goiania");  
4 System.out.println(cidades.get(0)); // Rio Verde  
5
```

Map (Chave-Valor):

```
1 Map<Integer, String> ddd = new HashMap<>();  
2 ddd.put(64, "Rio Verde");  
3 ddd.put(62, "Goiania");  
4 System.out.println(ddd.get(64)); // Rio Verde  
5
```

A Interface List

- É uma das interfaces mais utilizadas do Java Collections Framework.
- Representa uma coleção ordenada (sequência) de elementos.
- **Principais características:**
 - **Mantém a ordem de inserção:** Os elementos são armazenados na exata sequência em que foram adicionados.
 - **Permite elementos duplicados:** É possível ter o mesmo valor (ou mesmo objeto) repetido em posições diferentes.
 - **Acesso por índice:** Assim como os arrays, os elementos de uma lista podem ser acessados através de um índice numérico (começando em 0).



A interface `List` adiciona métodos específicos para lidar com índices, além dos já existentes na interface `Collection`:

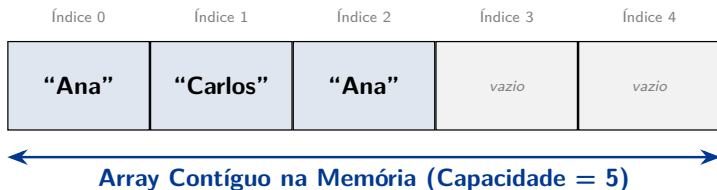
- `add(E e)` / `add(int index, E e)`: Adiciona um elemento.
- `remove(int index)` / `remove(Object o)`: Remove um elemento.
- `get(int index)`: Retorna o elemento na posição informada.
- `set(int index, E element)`: Substitui um elemento numa dada posição.
- `contains(Object o)`: Verifica se o elemento está na lista.
- `size()`: Retorna o número de elementos da lista.



- **ArrayList** é a implementação mais comum da interface `List`.
- Baseia-se em um array interno redimensionável.
- **Como funciona?**
 - Quando o array interno enche, ele cria um novo array maior e copia os elementos originais.
- **Vantagens:** Acesso muito rápido aos elementos via índice.
- **Desvantagens:** Inserções e remoções no meio da lista são custosas, pois exigem deslocamento de elementos.



Funcionamento do ArrayList



- **Acesso Direto (Índice):** É imediato. Como os elementos estão sequenciais na memória, o Java sabe exatamente onde encontrar o elemento do índice 2 sem precisar percorrer os anteriores.
- **Capacidade:** O array interno tem um tamanho fixo. Quando ele enche, o ArrayList cria automaticamente um novo array maior e move todos os elementos.

Exemplo Prático: ArrayList

```
1 import java.util.*;
2
3 public class Exemplo {
4     public static void main(String[] args) {
5         List<String> nomes = new ArrayList<>();
6
7         nomes.add("Ana");
8         nomes.add("Carlos");
9         nomes.add("Ana"); // Permite duplicados
10
11         System.out.println(nomes);
12     }
13 }
14
```

Saída:

[Ana, Carlos, Ana]



LinkedList

- A **LinkedList** implementa as interfaces `List` e `Deque` (Fila Duplamente Encadeada).
- Baseia-se em uma estrutura de **Lista Encadeada** interna.
- **Como funciona?**
 - Os elementos (nós) não são armazenados sequencialmente na memória. Cada nó contém o dado e referências para o nó anterior e posterior.
- **Vantagens:** Inserções e remoções frequentes (no início ou meio) são muito eficientes.
- **Desvantagens:** Acesso por índice é muito lento.





- **Encadeamento Duplo:** Cada nó é composto por uma referência para o elemento anterior (*prev*), o dado em si, e uma referência para o próximo (*next*).
- **Inserção/Remoção:** Para inserir um elemento no meio, basta alterar os ponteiros (ex: fazer o anterior apontar para o novo nó e o novo nó apontar para o próximo), o que é extremamente eficiente.

Exemplo Prático: LinkedList

```
1 import java.util.*;
2
3 public class ExemploFila {
4     public static void main(String[] args) {
5         LinkedList<String> fila = new LinkedList<>();
6
7         fila.add("Joao");
8         fila.add("Maria");
9
10        System.out.println(fila);
11    }
12 }
13
```

Saída:

[Joao, Maria]



Comparação: ArrayList vs LinkedList

Operação	ArrayList	LinkedList
Acesso por índice	Excelente	Ruim
Inserção no meio	Ruim	Boa
Remoção no meio	Ruim	Boa

- **Resumo:** Na maioria dos casos, o ArrayList oferece a melhor performance geral e deve ser a escolha padrão. Use LinkedList se sua aplicação tiver forte necessidade de realizar inserções e deleções constantes nas posições intermediárias.



A Interface Set

- A interface Set modela a estrutura matemática de conjuntos.
- **Características principais:**
 - **Não permite duplicatas:** Tentativas de adicionar um elemento repetido são ignoradas.
 - **Ideal para elementos únicos:** Útil para guardar CPFs, RGs, ou qualquer dado que requeira unicidade.
- A implementação mais comum e rápida é o **HashSet**.



Exemplo Prático: HashSet

```
1 import java.util.*;
2
3 public class ExemploHashSet {
4     public static void main(String[] args) {
5         Set<String> nomes = new HashSet<>();
6
7         nomes.add("Ana");
8         nomes.add("Pedro");
9         nomes.add("Ana"); // Ignorado
10
11         System.out.println(nomes);
12     }
13 }
14
```

Possível Saída:

[Pedro, Ana]

Observação: O elemento duplicado "Ana" não é armazenado. A ordem não é garantida.



TreeSet

- O **TreeSet** é outra implementação popular da interface **Set**.
- **Características:**
 - **Não aceita duplicados:** Assim como qualquer **Set**.
 - **Mantém ordenação automática:** Os elementos são organizados seguindo a sua ordem natural (alfabética para **String**, numérica para **Integer**) ou por um comparador fornecido.
- **Vantagem:** Buscas e iterações ocorrem sempre de forma ordenada.
- **Desvantagem:** As operações de inserção são um pouco mais lentas que o **HashSet**.



Exemplo Prático: TreeSet

```
1 import java.util.*;
2
3 public class ExemploTreeSet {
4     public static void main(String[] args) {
5         Set<Integer> numeros = new TreeSet<>();
6
7         numeros.add(30);
8         numeros.add(10);
9         numeros.add(20);
10
11         System.out.println(numeros);
12     }
13 }
14
```

Saída:

[10, 20, 30]



A Interface Map

- Um Map (Mapa/Dicionário) **não** é uma extensão da interface Collection.
- **Características principais:**
 - **Armazena pares chave-valor:** Cada elemento possui uma chave única que mapeia para um valor.
 - **Não permite chaves duplicadas:** Cada chave pode ter no máximo um valor associado.
 - Se você adicionar um novo valor para uma chave já existente, o valor antigo será sobrescrito.
- A implementação mais comum é o **HashMap**.



Exemplo Prático: HashMap

```
1 import java.util.*;
2
3 public class ExemploHashMap {
4     public static void main(String[] args) {
5         Map<Integer, String> alunos = new HashMap<>();
6
7         alunos.put(1, "Ana");
8         alunos.put(2, "Carlos");
9
10        System.out.println(alunos.get(1));
11    }
12 }
13
```

Saída:

Ana



- `put(K key, V value)`: Insere ou atualiza um par chave-valor.
- `get(Object key)`: Retorna o valor associado à chave.
- `remove(Object key)`: Remove o mapeamento para aquela chave.
- `containsKey(Object key)`: Verifica se o mapa contém a chave.
- `containsValue(Object value)`: Verifica se o mapa contém o valor.
- `keySet()`: Retorna um Set com todas as chaves.
- `values()`: Retorna uma Collection com todos os valores.

Ordenação de Collections

- Frequentemente precisamos organizar os dados em uma lista (por exemplo, ordem alfabética ou do menor para o maior).
- Estruturas como `ArrayList` e `LinkedList` mantêm apenas a ordem de inserção.
- Para ordená-las ativamente, o Java oferece métodos utilitários embutidos.
- A classe utilitária `java.util.Collections` contém algoritmos estáticos que operam sobre coleções.



O Método Collections.sort()

- O método `Collections.sort(List<T> list)` ordena a lista especificada na sua ordem natural ascendente.
- **Requisitos:**
 - Os elementos da lista devem implementar a interface `Comparable`.
 - Tipos básicos do Java como `String`, `Integer`, `Double` já implementam isso nativamente.
- É um algoritmo altamente otimizado.



Exemplo Prático: Ordenação

```
1 import java.util.*;
2
3 public class ExemploOrdenacao {
4     public static void main(String[] args) {
5         List<Integer> numeros = new ArrayList<>();
6
7         numeros.add(20);
8         numeros.add(5);
9         numeros.add(15);
10
11         Collections.sort(numeros);
12
13         System.out.println(numeros);
14     }
15 }
16
```

Saída:

[5, 15, 20]

